Biomedical Signal Processing and Machine Learning
Module 1 – Practical exam (AY 2023-24)
Second Cycle Degree/Two Year Master In Biomedical Engineering
9 September 2024

Setting up the development platform (Google Colab)

1. Go to eol.unibo.it, log in to today's session, and download the file **'Template_Module1_20240909.ipynb'.**
2. Go to https://colab.research.google.com/ , and log in at the top right with your Google account.
3. Upload the template to Google Colab, for example, by selecting **'File -> Upload Notebook -> Upload'**

Documentation

Directly use the following links for the Python documentation (DO NOT use Google or other search engines because of firewall settings):

- https://docs.python.org/3/
- https://pandas.pydata.org/docs/
- https://scikit-learn.org/stable/
- https://numpy.org/doc/
- https://matplotlib.org/stable/index.html

Exercise

The goal of this exercise is to understand the concepts of overfitting and underfitting in the context of polynomial regression. You will start with the provided synthetic regression data and fit it using polynomial models of increasing complexity. You will then visualize how the model's performance changes on the training set and test set as the polynomial degree increases.

Steps to Follow:

1. **Generate Synthetic Data**:
   - The synthetic dataset has already been generated for you using a sinusoidal function (sin(x)) with added Gaussian noise.
2. **Adoption of hold-out strategy and implementation of polynomial regression**:
   - Using the scikit-learn package, adopt a hold-out strategy (test set size = 30%).
   - To fit a polynomial regression model, you need to transform the input features into polynomial features. For this, use the PolynomialFeatures class from the sklearn.preprocessing module. Follow this code example to fit a polynomial regression model:

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# Example: Fitting a Polynomial Regression Model
```

```
# Choose the polynomial degree
degree = 3  # You can change this to explore different degrees

# Create polynomial features
poly_features = PolynomialFeatures(degree=degree)

# Transform the training data into polynomial features
X_train_poly = poly_features.fit_transform(X_train)

# Fit the linear regression model to the transformed features
model = LinearRegression()
model.fit(X_train_poly, y_train)

# Transform the test data using the same polynomial transformation
X_test_poly = poly_features.transform(X_test)

# Predict using the model
y_train_pred = model.predict(X_train_poly)
y_test_pred = model.predict(X_test_poly)
```

   - Fit polynomial regression models for degrees ranging from 1 to 15.
   - For each degree:
     - Calculate the mean squared error (MSE) on both the training set and the test set.
     - Store these errors for visualization.
3. **Visualize the results**:
   - Plot the mean squared error (MSE) for both the training set and the test set as a function of the polynomial degree.
4. **Interpret the resulting graph**:
   - Explain how the graph illustrates the concepts of overfitting and underfitting (write your answer in a separate Google Colab text cell).

Submitting the solution on eol.unibo.it

Save your Python code on your PC, rename it, and upload it to eol.unibo.it